CS104: Computer Architecture - CPU Simulator

Portfolio Project by Ando M | Codecademy CS104

📌 Overview

Python program yang mensimulasikan cara kerja CPU secara lengkap: Fetch, Decode, Execute, Memory Access, dan Writeback. Dibuat off-platform sesuai instruksi portfolio project.

🧠 Fitur (Sesuai Rubrik Codecademy)

- **MemoryBus**: Simulasi RAM 256 alamat, load initial values dari `data_input.txt`
- **Cache**: Direct-mapped cache 8-block, write-through, dengan hit/miss tracking
- **CPU**: 8 register (R0-R7, R0 hardwired 0), Program Counter, Fetch-Decode-Execute loop
- **ISA MIPS-like**: `ADD, SUB, MUL, ADDI, LW, SW, BEQ, J, HALT`
- **Console Output**: Mendokumentasikan setiap stage per cycle (wajib di tugas)
- **File I/O**: Baca instruksi dari `instruction_input.txt` & memory dari `data_input.txt`

🗂️ Struktur File

```
cpu_simulator/
├── cpu.py                # Class MemoryBus, Cache, CPU (inti projec
├── main.py               # Entry point versi modular (recommended)
├── main_single.py        # Versi single-file (backup, anti-import e
├── instruction_input.txt  # Daftar instruksi
├── data_input.txt        # Initial memory values
├── README.md
```

```
└── .gitignore
```

▶️ Cara Menjalankan

bash

```bash
python main.py
# atau jika ada masalah import:
python main_single.py
```

Contoh output:

```
[FETCH] PC=2 | ADD,R3,R1,R2
[DECODE] Operation: ADD, Args: ['R3', 'R1', 'R2']
[EXECUTE] R3 = R1(10) + R2(20) = 30
[MEMORY] SW MEM[0] <- R3 = 30
[WRITEBACK] Registers: {...}
...
SIMULATION FINISHED
Total Cycles: 11
Final Registers: {'R0': 0, 'R1': 10, 'R2': 20, 'R3': 30, ...}
Cache Hits: 1, Misses: 0, Hit Rate: 100.0%
```

🔧 Git Workflow (Project Objective)

bash

```bash
git init
git add .
git commit -m "Complete CS104 CPU Simulator"
```

```
git push origin main
```

💡 Yang Saya Pelajari

- Cara kerja CPU pipeline
- Konsep cache hit/miss
- File I/O dan parsing instruksi
- Git & command line development

Submitted for Codecademy CS104 Portfolio Project